

8개리그 · EPL 예측 앱

파이프라인/프론트엔드 작업 인수인계 문서

2026-08-01 세션 종료 시점 기준 (직전 세션 포함 누적 정리)

이 문서의 목적: 채팅방 용량 한계로 새 채팅방에서 이어서 작업해야 하는 상황입니다. 아래 "완료 (재검토 불필요)" 항목은 실제 로그/스크린샷/유닛테스트로 검증이 끝난 것들이므로 새 채팅방에서 다시 확인하거나 의심할 필요 없이 그대로 신뢰하고 이어서 작업하면 됩니다. "△ 부분검증"과 "진행중/미완료" 항목만 다음 세션에서 마저 처리하면 됩니다.

△ 매우 중요 — 다음 채팅방 시작 시 필수 절차

이 PDF는 코드 내용 자체를 담고 있지 않습니다(코드 파일은 별도로 첨부됩니다). 새 채팅방을 열면 이 PDF와 함께, 아래 "5. 다음 채팅방에 올려야 할 파일 목록"에 있는 파일들을 전부 업로드하고 시작해야 합니다. 파일 없이는 이어서 작업 불가능합니다.

1. 프로젝트 개요

구조: 완전히 별도인 GitHub Pages 앱 2개.

- EPL 앱: EPL26-27 저장소 → reports/EPL_index.html → style2580n - droid.github.io/epl/
- 8개리그 앱: 별도 저장소 → multi_league_index.html → style2580n - droid.github.io/8leagues/ (라리가 · 분데스리가 · 세리에A · 리그앙 · 에레디비시 · 챔피언십 · MLS · 엘리트세리엔)
- 파이프라인 저장소: epl - management — football_pipeline.yml이 GitHub Actions로 매일 실행, Python 수집기들이 순서대로 돌면서 data/ 아래 JSON/DB 파일을 만들고, app_export*.py가 최종 HTML에 데이터를 임베드함.

이번 세션 작업의 큰 줄기: (1) API - FOOTBALL 유료화로 끊긴 선수 개인지표(슈팅/키패스/태클 등)를 BSD API로 대체 수집 → (2) 그 과정에서 파이프라인 실행 순서 버그, 파싱 버그, 필드명 불일치를 연쇄로 발견/수정 → (3) 카드 누적 정지, 프리시즌 친선전 지원, 리더보드 확장 등 신규 기능 추가.

2. 완료 — 실제 검증 완료, 재검토 불필요

아래 항목들은 유닛테스트 통과 + 실제 파이프라인 로그 확인 또는 사용자 스크린샷으로 화면 동작 확인까지 끝난 것들입니다. 다시 의심하거나 재검증하지 말고 그대로 두고 다음 작업으로 넘어가면 됩니다.

2-1. 파이프라인 순서 버그 (이번 세션 최대 원인 규명)

[근본원인] db.py가 collect_fixtures.py보다 먼저 도는 구조라, h2h/날짜/진짜 BSD id 백필이 매번 '예전' 데이터 기준으로 이뤄지고 있었음(순환 지연 1턴).

- 해결: collect_fixtures.py/collect_fixtures_multileague.py 직후에 db.py + normalizer.py 재실행을 football_pipeline.yml에 추가하고, collect_bsd_player_stats.py를 그 뒤로 이동시킴.
- player_profiler.py/season_aggregator.py/transfer_impact.py/report_generator.py도 app_export*.py 직전에 재실행 추가(최신 데이터 반영 위함 — 안 그러면 이 스크립트들도 예전 데이터를 봄).
- 검증: YAML 파싱 정상 + 순서 로직 코드검증 + 실제 파이프라인 로그로 순서 확인 완료. db.py를 2번 돌리면 canonical_id(정규화된 선수ID)가 리셋되는 부작용도 발견해서 normalizer.py 재실행으로 같이 해결(link_events())가 NULL인 것만 재처리하도록 이미 설계돼 있어서 안전).

2-2. collect_fixtures.py 루프변수 스코프 버그

[버그] schedule.json의 id 필드가 385건 전부 똑같은 값(159793)이었음 — 파이썬 for루프 변수가 루프 종료 후에도 스코프에 남는 특성 때문에 생긴 버그.

- 실측 확인(schedule.json 직접 파싱해서 유니크값 개수 세어봄 → 1개였음) 후 수정.
- 8개리그판(collect_fixtures_multileague.py)은 원래부터 이 버그가 없었음(다른 코드 패턴) — EPL판만 문제였음.
- 검증: 재실행 후 385건 전부 유니크 id 확인 완료.

2-3. db.py 개선

- matches 테이블에 bsd_event_id 컬럼 추가(진짜 BSD 숫자id). 기존 match_id는 "홈팀_원정팀_숫자" 형태의 합성id라 BSD 진짜 id가 아니었음 — 이걸 혼용해서 collect_bsd_player_stats.py가 전부 404를 받고 있었던 게 근본 원인 중 하나였음.
- h2h(상대전적) lookup이 날짜뿐 아니라 진짜 BSD id도 같이 반환하도록 확장.
- load_fixtures_openfootball() 함수가 새 9컬럼 스키마를 놓쳐서 SQL 에러가 날 뻔한 걸 발견하고 수정.
- 구버전 DB 파일에도 안전하게 컬럼이 추가되도록 마이그레이션(ALTER TABLE) 로직 추가.
- 검증: end - to - end 시뮬레이션(mock DB로 전체 흐름 실행) + 실제 파이프라인 로그로 확인 완료.

2-4. collect_bsd_player_stats.py — 핵심 수집기 (가장 많이 고친 파일)

- match_id(합성id) 대신 진짜 bsd_event_id로 조회하도록 수정.
- 실제 응답 파싱 버그 수정: 최상위 키가 results가 아니라 player_stats, 드리블 필드가 dribble_attempted/dribble_won이 아니라 total_contest/won_contest (전부 실제 로그로 실측 확인).
- 응답에 선수 '이름'이 없고 player_id(숫자)만 있는 걸 발견 → /api/v2/players/{id}/ 로 이름을 별도 조회하고 캐시(data/master/bsd_player_names_cache.json)에 저장해서 재사용 — 캐시가 쌓일수록 실행이 빨라지는 구조.
- tackles_won / tackles 필드명을 이중 저장 — app_export.py는 tackles_won을, player_profiler.py 등 기존 파일들은 tackles를 기대해서 코드베이스 안에 두 관례가 공존했음(실측 확인).
- STATE_VERSION 도입 — 예전 파싱버그 시절에 잘못 'no_data'로 마킹해둔 243건이 그대로 남아있으면 파싱을 고쳐도 재시도가 영원히 안 되는 문제를 발견 → 버전이 바뀌면 done_matches를 자동 리셋.
- 리더보드 확장용 필드 대량 추가: saves, goals_conceded, clearances, fouls, was_fouled, touches, total_pass, accurate_pass, total_long_balls, accurate_long_balls, total_cross, accurate_cross, duel_won, duel_lost, aerial_won, aerial_lost, ball_recovery, blocked_scoring_attempt, minutes_played.
- 검증: rule - 3 테스트 27건+ 통과. 실제 파이프라인 로그로 14경기(73명)→15경기(85명) 추가 병합 확인, 카드 결장위험 판정이 실전에서 실제로 작동하는 것(레드카드 직전경기 선수 1명 발견)까지 확인.

2-5. collect_api_football_player_stats.py

tackles_won/tackles 이중저장 추가(위 2 - 4와 동일 이유). 검증: 문법 검증 통과.

2-6. compute_card_suspensions.py (신규 스크립트)

- 경기별 yellow_cards/red_cards를 모아서 시즌누적/직전경기 레드카드로 결장위험을 판정.
- 처음엔 "옐로 5장 이상 = 경고" 단순 로직이었으나, 사용자가 직접 조사해준 9개 리그(EPL · 챔피언십 · 라리가 · 세리에A · 분데스리가 · 리그앙 · 에레디비시 · 엘리트세리엔 · MLS)의 정확한 누적 정지 규정을 그대로 반영해서 전면 재작성함(리그마다 임계값 패턴이 다 다름 — 예: 라리가/세리에A는 5→10→14→17→19장, 그 이후 매장마다).
- 레드카드는 "직전 경기 퇴장 → 다음 경기 결장 확정"급으로, 옐로누적은 리그 규정에 정확히 도달한 시점만 "정지 위험" 경고로 구분해서 표시.
- 알려진 한계(문서화됨): 경기수 제한(예: "19경기 이내 5장") 미반영, 컵대회 카드 미분리, 시즌중 리셋(에레디비시/MLS) 미반영, 정지 소화 여부 추적 불가(근사치).
- 검증: 23개 테스트 통과(1~25장까지 전체 시퀀스가 9개 리그 전부 사용자 제공 규정과 정확히 일치 확인). 실제 파이프라인에서 결장위험 선수 실제로 발견됨(레드카드 케이스) 확인.

2-7. 프리시즌(친선전) 지원 — collect_fixtures.py / collect_fixtures_multileague.py

- BSD의 "Club Friendlies" 리그를 실제 검색으로 발견(ID 하드코딩 아님 — 매번 리그 목록에서 이름 매칭).
- EPL+8개리그 전체 팀명을 아우르는 매칭 함수를 새로 만들어서, 친선전 상대가 다른 리그 소속(예: EPL팀 vs 라리가팀 친선전)이어도 인식 가능하게 함.
- 종료된 친선전은 상대전적(h2h)에서 제외(진지도 낮은 경기라 통계 왜곡 방지) — 예정된 친선전만 일정에 반영.
- friendly 플래그 추가해서 프론트엔드가 정규시즌/친선전을 구분 표시 가능하게 함.
- 검증: 실제 파이프라인 로그로 확인(EPL 3960건 발견, 8개리그는 리그별로 정확히 배분된 것까지 확인 — 챔피언십11 · 리그1 32 · 분데스리가35 · 세리에A34 · 에레디비시6 · 라리가34 · MLS1건). 스크린샷으로 화면에 "친선" 노란 뱃지가 정상 표시되는 것까지 확인.

2-8. 프론트엔드 — 확실히 검증된 것들 (스크린샷/사용자 확인 완료)

- 일정 화면에 "친선" 뱃지 표시 — 스크린샷으로 확인.
- 카드 결장위험을 "부상/결장" 패널에 표시 — 실제 파이프라인 로그로 데이터 흐름 확인.
- 핵심선수 자동체크(부상/카드 결장 선수는 수동 체크 없이 자동으로 체크됨, 승률에 즉시 반영) — 스크린샷으로 실제 작동 확인(선수 2명이 자동으로 체크되고 취소선 표시된 것까지 확인).

- getTeamRadarProfile 크래시 버그 수정 — 프리시즌 낮선 상대팀(예: SC 캄뷔르 같은 팀) 선택 시 "Cannot read properties of undefined (reading '공격')" 에러로 화면 전체가 죽던 문제. 원인: ELO 데이터 없는 팀에 대해 undefined 반환하던 버그. 수정 후 사용자가 시크릿모드로 재확인해서 "작동은 다되네"로 최종 확정 — 이전 에러는 브라우저 캐시 문제였고 코드 자체는 정상 확인됨.
- TeamStyleRadarCard를 ErrorBoundary로 추가 감싸서, 혹시 모를 다른 원인의 크래시도 그 카드 하나만 죽고 전체 화면은 안 죽도록 안전장치 추가.
- 카드 결장정보를 "라인업 1시간 전부터 채워짐" 문구와 분리 — 사용자 지적("과거 기록 기반 데이터인데 왜 라인업 타이밍에 묶여있냐")이 정확했음, 8개리그 앱에서 분리 완료.
- 오버/언더 3.5 추가, 최고확률 스코어 강조 표시 — 8개리그 앱에 추가함(EPL 앱은 이미 둘 다 구현되어 있었음 — 확인 완료, 사용자가 본 스크린샷은 8개리그 앱이었음).

3. ⚠️ 부분검증 — 코드/테스트는 통과했지만 실제 화면 · 파이프라인 결과는 아직 미확인

아래는 코드 작성과 문법검증, 일부는 mock 데이터 실행검증까지 끝났지만, 실제 파이프라인을 돌려서 화면으로 확인하는 마지막 단계가 아직 안 된 것들입니다. 다음 세션에서 파이프라인 실행 → 결과 확인만 하면 됩니다(코드를 다시 짤 필요는 없음).

3-1. 로고 — collect_logos_multileague.py / app_export.py / app_export_multileague.py

- 정규 8개리그+EPL 소속이 아닌 친선전 상대팀 로고도 TheSportsDB로 수집하도록 확장(_friendly 키).
- EPL 앱은 원래 로고 시스템이 아예 없었음(20개 팀 하드코딩 로컬 이미지 경로뿐) → 원격 로고 폴백 추가.
- ⚠️ 중요 발견: 스크린샷에서 로고가 안 보이던 팀들(SC 캄뱌르, 우디네세 등) 상당수를 조사해보니 실제로는 이미 정규 8개리그 팀 목록에 있는 팀들이었음 — "친선전이라 목록 밖"이 아니라, 최근(7/25) 추가된 16개 팀의 로고 수집 자체가 아직 안 됐거나 실패했을 가능성이 더 높음. 다음 세션에서 collect_logos_multileague.py 실행 로그를 반드시 확인해서 진짜 원인을 규명해야 함.
- 코드 테스트(친선전 진짜 낯선팀 매칭 로직)는 통과했지만, 실제 배포 후 화면 확인은 안 됨.

3-2. collect_goalscorers.py — 헤더골/세트피스골 데이터 확장

- 골 이벤트 원문에서 goal_type(regular/penalty 등)과 body(sequence 마지막 동작의 신체부위)를 추출하도록 확장.
- 실측 확인됨(추측 아님): 실제 이전 파이프라인 로그에서 골 이벤트에 goal_type='regular', body='right - foot' 필드가 실제로 존재하는 걸 확인함.
- ⚠️ 미확정: '헤더'(head)의 정확한 표기값은 아직 실측 안 됨 — 지금까지 로그에서는 'right - foot' 하나만 나왔음(원발/헤더는 아직 안 나옴). 진단 로그(관측된 body 값 전체 출력)를 이미 코드에 추가해줬으니, 다음 파이프라인 실행 로그에서 [diag] 관측된 body 값 전체를 확인해서 'head' 표기가 맞는지, 혹시 'header'나 다른 표기인지 검증 필요.
- 검증: rule - 3 테스트 10건 통과(실제 로그에서 나온 진짜 데이터를 그대로 사용해서 테스트함).

3-3. build_leaderboard 확장 — app_export.py / app_export_multileague.py

- 득점왕/도움왕 외에 슈팅 · 유효슈팅 · 키퍼스 · 드리블 · 드리블성공 · 태클 · 인터셉트 · 클리어링 · 선방 · 경고 · 퇴장 · 헤더골 · 페널티골 · 프리킥골 · 출전경기 · 출전시간까지 확장.
- season_players.json(collect_bsd_player_stats.py가 채운 필드들이 season_aggregator.py를 거쳐 이미 누적돼있는 파일)을 그대로 활용 — 별도 배선 불필요, 필드만 뽑으면 됨.
- gk/cards 필드를 프론트엔드가 기대하는 객체 구조({saves,cs,apps} / {yellow,red})로 재구성 (처음엔 단순 숫자로 잘못 만들어서 구조 불일치 버그가 있었음 — 발견하고 수정함).
- 검증: mock 데이터로 실제 함수 실행까지 확인(리그별로 정확히 분리되는 것까지 확인 — 예를 들어 라리가 리더보드에 EPL 선수가 안 섞이는 것 확인). 단, 실제 파이프라인 실행 후 결과는 아직 미확인.

3-4. 프론트엔드 리더보드 화면 확장

- EPL 앱: 리더보드 화면에 헤더골/페널티골/프리킥골/출전경기/출전시간 탭 추가. (참고: 슈팅/키퍼스/태클/인터셉트/선방 탭은 이미 예전부터 있었음 — 확인 완료. 처음에 "다른 게 다 빠졌다"고 하신 건 8개리그 앱 얘기였을 가능성이 높음)
- 8개리그 앱: 득점왕/도움왕 2개뿐이던 리더보드를 17개 카테고리(득점/도움/헤더골/페널티골/프리킥골/슈팅/유효슈팅/키퍼스/드리블/태클/인터셉트/클리어링/선방/경고/퇴장/출전경기/출전시간)로 전면 확장. 모바일 가로스크롤 탭 형태로 구현.
- ⚠️ 문법검증만 완료 — 이번 세션 마지막에 작업해서 실제 파이프라인 실행 후 화면 확인이 전혀 안 된 상태. 다음 세션에서 제일 먼저 확인해야 함.

4. 진행중 / 시작 전 — 다음 세션에서 처리해야 할 것

- [미착수] 각 팀 로스터(스쿼드) 화면에 개인 통계(슈팅/키패스/테클 등) 표시 — 사용자가 요청했으나 이번 세션에 시작도 못 함.
- [확인필요] 로그 안 나오는 근본 원인 규명 — 위 3 - 1 참고, 다음 파이프라인 로그 확인 필요.
- [확인필요] 헤더골 body='head' 표기 실측 확인 — 위 3 - 2 참고, 다음 파이프라인 [diag] 로그 확인.
- [확인필요] 8개리그 리더보드 확장 실제 화면 동작 확인 — 위 3 - 4 참고.
- [확인필요] build_leaderboard 실제 파이프라인 실행 결과 확인 — 위 3 - 3 참고.

권장 순서: (1) 아래 5번 파일들을 전부 저장소에 올리고 파이프라인 1회 실행 → (2) 로그에서 [diag] 표시들(특히 collect_logos_multileague.py, collect_goalscorers.py의 body 값) 확인 → (3) 앱 화면에서 리더보드/로그/카드결장 정상 동작 확인 → (4) 문제 있으면 로그와 함께 보고 → (5) 팀 로스터 화면 통계 표시 작업 시작.

5. 다음 채팅방에 올려야 할 파일 목록

이 세션에서 최종 수정된 파일들(전부 이번 대화의 마지막 결과물로 함께 전달됨) — 다음 채팅방을 열면 이 13개 파일을 전부 업로드하고 시작해야 합니다.

파일명	역할	상태
collect_fixtures.py	EPL 일정/상대전적 수집 (친선전 지원 포함)	완료
collect_fixtures_multileague.py	8개리그 일정/상대전적 수집 (친선전 지원 포함)	완료
db.py	SQLite DB 적재 (bsd_event_id 컬럼 포함)	완료
collect_bsd_player_stats.py	선수 개인지표 수집 (핵심 파일)	완료
collect_api_football_player_stats.py	API - FOOTBALL 개인지표 보조수집	완료
compute_card_suspensions.py	카드 누적 결장위험 판정 (신규)	완료
collect_logos_multileague.py	구단 로고 수집 (친선전 상대팀 포함)	부분검증
collect_goalscorers.py	득점자/헤더골/세트피스골 수집	부분검증
football_pipeline.yml	GitHub Actions 파이프라인 정의	완료
app_export.py	EPL 앱 데이터 임베드 (리더보드 확장 포함)	부분검증
app_export_multileague.py	8개리그 앱 데이터 임베드 (리더보드 확장 포함)	부분검증
EPL_index.html	EPL 앱 프론트엔드	부분검증
multi_league_index.html	8개리그 앱 프론트엔드	부분검증

'완료' 표시 파일: 실제 파이프라인 로그로 정상 작동까지 확인됨 — 다음 세션에서 코드 재검토 불필요, 그대로 신뢰하고 사용.

'부분검증' 표시 파일: 코드 자체는 테스트 통과했지만 실제 파이프라인 결과 확인이 남음 — 다음 세션 시작하면 제일 먼저 파이프라인 1회 실행하고 로그부터 확인할 것.

추가로 필요할 수 있는 파일 (이번 세션엔 요청 안 했지만, 작업 중 참조된 파일들)

- 이번 세션에서는 위 13개 파일 외에 추가로 받은/수정한 백엔드 파일은 없습니다.
- 단, 팀 로스터(스쿼드) 화면 작업을 이어가려면 EPL_index.html / multi_league_index.html 안의 SquadTab 관련 코드를 다시 봐야 하는데, 이건 이미 위 13개 파일 안에 포함되어 있으므로 추가로 받을 파일은 없습니다.
- 혹시 다음 세션에서 "collect_goalscorers.py의 body='head' 확인이 안 된다"거나 "로고가 여전히 안 나온다" 같은 문제가 재발하면, 가장 최근 파이프라인 실행 로그 zip 파일을 요청하게 될 가능성이 높습니다 — 미리 준비해두면 빠릅니다.

이 문서는 2026-08-01 세션 종료 시점까지의 작업 내용을 정리한 것입니다. 새 채팅방에서 이 PDF와 위 13개 파일을 함께 첨부하고 "인수인계 문서 참고해서 이어서 작업해줘"라고 요청하면 바로 이어서 진행할 수 있습니다.